

ASIC Design & Verification on a Reproducible Workbench

Day 2 — RTL Simulation and Logic Synthesis, End-to-End on Crucible

Simulate a 4-bit RISC-style counter, view its waveform, and synthesize an RTL design into a standard-cell gate-level netlist — every step driven by the Crucible CLI (`ign`), in one reproducible workspace, with no manual tool installation.

#	EXPERIMENT	TOOLS (VIA <code>ign</code>)	RESULT
1	RTL simulation + waveform	iverilog, vvp, gtkwave	✓ PASS
2	Logic synthesis → gate netlist	yosys + Nangate .lib	✓ PASS

T A B L E O F C O N T E N T S

01	Executive Summary	3
02	The Reference Design	4
03	Crucible Workspace Setup	4
04	Experiment 1 — RTL Simulation and Waveform	5
05	Experiment 2 — Logic Synthesis into a Gate-Level Netlist	7
06	Conclusion	8

EXECUTIVE SUMMARY

Executive Summary

Hardware verification and synthesis normally depend on a hand-assembled toolchain — a simulator, a waveform viewer, a synthesis engine, and a standard-cell library — each installed and version-matched by hand. That setup is fragile and rarely reproduces across machines.

This lab note demonstrates the same flow run on a managed workbench. Using Crucible (igny), we stand up an isolated environment, install three catalogued open-source tools, and run a complete Day 2 ASIC flow: simulate a 4-bit counter against its testbench, view the resulting waveform, and synthesize an RTL design into a gate-level netlist mapped to the Nangate standard-cell library — all without installing a single tool by hand.

Two experiments, both passing, both run entirely through igny:

#	EXPERIMENT	TOOLS (VIA igny)	RESULT
1	RTL simulation + waveform counter verified, count.vcd produced	iverilog, vvp, gtkwave	✓ PASS
2	Logic synthesis → gate netlist synth_example.v produced	yosys + Nangate .lib	✓ PASS

SECTION 1

The Reference Design

Two small, self-contained Verilog designs — one for simulation, one for synthesis.

SYSTEM SPECIFICATION

COMPONENT	DETAIL
DUT (sim)	Mycounter — 4-bit synchronous up-counter with synchronous reset
Testbench	Testbench.v — clock/reset stimulus, VCD dump, \$monitor console trace
DUT (synth)	top — 2:1 multiplexer feeding a clocked register
Cell library	NangateOpenCellLibrary_typical.lib (open 45 nm, for mapping)
Workbench	Crucible CLI (igny), Community tier

SECTION 2

Crucible Workspace Setup

All Day 2 tools — iverilog (which provides vvp), gtkwave, and yosys — are in the Crucible catalog. One environment, one workspace, three installs:

```
# activate the Crucible sandbox igny init
igny env create day2_sim
igny workspace create --path . --env day2_sim
cd day2_sim
# install all three tools igny tool install iverilog
igny tool install gtkwave
igny tool install yosys
igny tool list
# confirm installed + bound
```

How igny run works: arguments after the binary name are passed verbatim to the tool. Because iverilog declares provides = ["iverilog", "vvp"], igny run vvp ... works with no separate install. Always run from the workspace root (the folder containing crucible.toml).

SECTION 3

Experiment 1 — RTL Simulation and Waveform

OBJECTIVE

Compile a 4-bit counter against its testbench, run the simulation, and confirm correct count-and-reset behaviour on the waveform.

DESIGN UNDER TEST — MYCOUNTER.V

```
module Mycounter(CLK, RST, OUT);
  input  CLK, RST;
  output [3:0] OUT;
  reg    [3:0] OUT;
  always @(posedge CLK) begin
    if (RST == 1'b1) OUT <= 4'b0000;
    else OUT <= OUT + 1;    // count up, wraps 15 → 0
  end
endmodule
```

STIMULUS — TESTBENCH.V

TIME (NS)	EVENT
0	Clock=0, Reset=1 — counter held in reset
100	Reset=0 — counting begins
2100	Reset=1 — counter forced back to 0
2500	\$finish — simulation ends

Clock toggles every 50 ns (always #50 Clock=~Clock) → period = 100 ns. \$dumpfile("count.vcd") + \$dumpvars capture all signals; \$monitor prints time, Clock, Reset, Count on every change.

SECTION 3 (CONT.)

PROCEDURE

```
# 1. compile → Mycounter igny run iverilog -o Mycounter Mycounter.v Testbench.v
# 2. simulate → count.vcd + console log igny run vvp Mycounter
# 3. view waveform igny run gtkwave count.vcd
```

RESULTS

- The `$monitor` console trace shows Count holding at 0 while Reset=1, then incrementing 0, 1, 2, ... , 15 and wrapping, until the reset at t=2100 drives it back to 0.
- vvp terminates with Testbench.v:15: `$finish` called at 2500.
- Artifacts produced: Mycounter (compiled simulation) and count.vcd.
- GTKWave renders OUT[3:0] ramping 0→15 repeatedly between the two reset pulses; both reset edges drive it to 0.

Experiment 1 — PASS



The write-compile-simulate-view cycle runs end-to-end through igny . The counter behaves exactly as specified.

ARTIFACTS PRODUCED

Mycounter

Compiled simulation binary (iverilog output)

count.vcd

Value Change Dump — waveform data for GTKWave

SECTION 4

Experiment 2 — Logic Synthesis into a Gate-Level Netlist

OBJECTIVE

Synthesize the RTL design `top.v` into a gate-level netlist mapped to the Nangate standard-cell library, using Yosys.

DESIGN TO SYNTHESIZE — TOP.V

```
module top(a, b, clk, select, out);

input  a, b, clk, select;

output out;

reg    out;

wire   y;

assign y = (select) ? b : a; // 2:1 multiplexer (combinational)    always @(posedge clk)
    out <= y;
// register → maps to a D flip-flop endmodule
```

Synthesis converts this behavioural RTL into real cells: the `assign` becomes gate logic; the `always @(posedge clk)` becomes a D flip-flop.

SYNTHESIS SCRIPT — YOSYS_COMMANDS.TCL

STEP	COMMAND	ACTION
1	<code>read_verilog top.v</code>	Parse the RTL source
2	<code>hierarchy -check -top top</code>	Elaborate hierarchy, set the top module
3	<code>proc</code>	Convert always/if/case processes into muxes + registers
4	<code>techmap</code>	Map to Yosys's internal generic gate library
5	<code>dfflibmap -liberty ...Nangate...lib</code>	Map flip-flops to Nangate library cells
6	<code>abc -liberty ...Nangate...lib</code>	Optimize + map combinational logic to Nangate cells
7	<code>clean</code>	Remove unused cells and wires
8	<code>write_verilog -noattr synth_example.v</code>	Emit the gate-level netlist

PROCEDURE

```
# synthesise → synth_example.v igny run yosys -s yosys_commands.tcl
```

RESULTS

- Yosys prints per-step statistics; after `dfflibmap/abc` it reports the cell count and area of the mapped design.
- `synth_example.v` expresses the design purely in Nangate standard cells — a D flip-flop plus the gates implementing the 2:1 mux — instead of behavioural RTL.

**Experiment 2 — PASS**

`top.v` is synthesized into a Nangate-mapped gate-level netlist.

SECTION 5

Conclusion

#	CAPABILITY DEMONSTRATED	ARTIFACT	RESULT
1	RTL simulation + waveform on a virtual workbench	counter verified, count.vcd	✓ PASS
2	RTL → standard-cell gate-level netlist	synth_example.v	✓ PASS

Day 2 takes an RTL design through simulation → waveform inspection → logic synthesis, the core front-to-mid steps of an ASIC flow — and runs the whole thing through a single Crucible workspace. No tool is installed by hand. The gate-level netlist (synth_example.v) is the hand-off point into static timing analysis (Day 3) .

FULL COMMAND SEQUENCE

```
# workspace setup igny init
igny env create day2_sim
igny workspace create --path . --env day2_sim
cd day2_sim
igny tool install iverilog
igny tool install gtkwave
igny tool install yosys
# experiment 1 - rtl simulation igny run iverilog -o Mycounter Mycounter.v Testbench.v
igny run vvp Mycounter
igny run gtkwave count.vcd
# experiment 2 - logic synthesis igny run yosys -s yosys_commands.tcl
```